React Full Stack Developer

Interview Questions & Answers

With real-time examples, code snippets and MNC-style questions

Level	2 to 3 years of experience (Full Stack / React)
Stack covered	JavaScript, React 18, TypeScript, Node.js, Express, SQL, MongoDB, Redis, Docker, Git, AWS, Python/FastAPI and AI basics
Edition	October 2026

How to use this guide

- Each question has a difficulty tag: **Basic**, **Intermediate** or **Advanced**. Start with Basic and Intermediate, then add Advanced for senior or product-company interviews.
- Green boxes show a **real-time example** from e-commerce, dashboards or interview platforms. Use them to make your own answers practical.
- Do not memorise code. Understand the idea, then explain it in your own words and tie it to your project.
- Type the code snippets yourself in an editor at least once.

Note on MNC questions

These are the **types of questions commonly asked** in full stack and React interviews at MNCs such as TCS, Infosys, Wipro, Cognizant, Accenture, Capgemini, HCLTech, Amazon, Microsoft, Adobe, Walmart and Flipkart. They are not official or leaked question papers, and every company and panel is different. For a specific company, also read recent interview experiences on AmbitionBox, Glassdoor and GeeksforGeeks.

Contents

1. JavaScript Fundamentals	10 questions
2. React Core and Hooks	19 questions
3. TypeScript	5 questions
4. Node.js, Express and APIs	10 questions
5. Databases: SQL, MongoDB and Redis	8 questions
6. Docker, Git, CI/CD and AWS Basics	4 questions
7. Python and AI Basics (HR-Agent Project)	6 questions
8. Project and Scenario Questions	6 questions
9. Coding Round and Machine Coding	8 questions
10. HR and Behavioural Round	7 questions
11. Quick Revision: Cheat Sheets and Interview Tips	tables

Total: 83 questions with answers.

1. JavaScript Fundamentals

JavaScript questions open almost every technical round, in service MNCs and product companies alike. Expect output-based questions and a small coding task.

Q1. What is a closure? Where do you use it? [Basic]

A closure is a function bundled together with references to the variables of the scope in which it was created. The inner function keeps access to those variables even after the outer function has finished executing.

```
function createCart() {
  let items = [];                 // private, cannot be touched from outside
  return {
    add: (item) => items.push(item),
    count: () => items.length,
  };
}
const cart = createCart();
cart.add('shoe');
console.log(cart.count()); // 1
```

Real-time example: Closures give data hiding, memoization and event handlers. A classic React bug is the **stale closure**: a `setInterval` inside `useEffect(..., [])` keeps reading the first-render state. Fix it with the functional update `setCount(c => c + 1)` or correct dependencies.

Q2. Difference between var, let and const? What is hoisting and the Temporal Dead Zone? [Basic]

- `var` is function-scoped, hoisted and initialised as `undefined`.
- `let` and `const` are block-scoped. They are hoisted but stay in the Temporal Dead Zone (TDZ) until the declaration line, so reading them early throws a ReferenceError.
- `const` stops re-assignment of the binding, not mutation of the object it points to.

```
console.log(a); // undefined (var hoisted)
var a = 10;
console.log(b); // ReferenceError (TDZ)
let b = 20;
const user = { name: 'A' };
user.name = 'B';   // allowed
// user = {};      // TypeError
```

Real-time example: Loop bug: `for (var i = 0; i < 3; i++) setTimeout(() => console.log(i))` prints 3, 3, 3. With `let` it prints 0, 1, 2 because every iteration gets a new binding.

Q3. == vs ===? Which values are falsy? [Basic]

`==` performs type coercion before comparing; `===` compares value and type without coercion. Prefer `===`. The falsy values are `false`, `0`, `''`, `null`, `undefined` and `NaN`.

```
console.log(0 == '0');          // true
console.log(0 === '0');         // false
console.log(null == undefined); // true
console.log(NaN === NaN);       // false -> use Number.isNaN(x)
```

Real-time example: A query-string value such as `?page=2` is a string, so `page === 2` is false. Convert with `Number(page)` first.

Q4. Explain the event loop. What is the output? [Intermediate]

JavaScript is single-threaded. The call stack runs synchronous code; async callbacks wait in queues. When the stack is empty, the event loop first drains the **microtask queue** (Promise `.then`, `queueMicrotask`, async/await continuations) and only then takes one task from the **macrotask queue** (`setTimeout`, `setInterval`, I/O, UI events). In Node.js, `process.nextTick` runs even before promise callbacks.

```
console.log('1');
setTimeout(() => console.log('2'), 0);
Promise.resolve().then(() => console.log('3'));
console.log('4');
// Output: 1 4 3 2
```

Real-time example: A heavy `for` loop freezes the UI because it blocks the call stack, so clicks and rendering cannot run. Split the work into chunks or move it to a Web Worker.

Q5. Promise vs async/await? Promise.all vs allSettled vs race? [Intermediate]

`async/await` is syntax sugar over promises that reads like synchronous code and uses `try/catch` for errors. `Promise.all` resolves when all succeed and rejects on the first failure. `Promise.allSettled` waits for all and reports each result. `Promise.race` settles with the first promise to settle.

```
async function loadProductPage(id) {
  try {
    const [product, reviews, related] = await Promise.all([
      fetch(`/api/products/${id}`).then(r => r.json()),
      fetch(`/api/products/${id}/reviews`).then(r => r.json()),
      fetch(`/api/products/${id}/related`).then(r => r.json()),
    ]);
    return { product, reviews, related };
  } catch (err) {
    console.error('Page load failed', err);
  }
}
```

Real-time example: Parallel calls cut total time from the sum of three requests to the slowest one. If reviews are optional use `allSettled` so one failure does not break the page. Remember that `fetch` does not reject on HTTP 404/500, so always check `response.ok`.

Q6. How does `this` work? call vs apply vs bind? How are arrow functions different? [Intermediate]

`this` depends on how a function is called, not where it is written. `call(ctx, a, b)` invokes immediately with listed arguments, `apply(ctx, [a, b])` takes an array, and `bind(ctx)` returns a new function with `this` fixed. Arrow functions have no own `this`; they use the surrounding lexical `this`.

```
const user = { name: 'Mounika', greet() { return 'Hi ' + this.name; } };
const fn = user.greet;
fn();                       // this is lost (undefined or window)
fn.call({ name: 'Ravi' });  // 'Hi Ravi'
const bound = user.greet.bind(user);
bound();                    // 'Hi Mounika'
```

Real-time example: Class-component event handlers needed `this.handler = this.handler.bind(this)`. Function components with hooks and arrow functions removed that problem.

Q7. Debounce vs throttle. Implement both. [Intermediate]

Debounce waits until the user stops triggering the event for N ms. **Throttle** lets the function run at most once every N ms.

```
function debounce(fn, delay) {
  let timer;
  return function (...args) {
    clearTimeout(timer);
```

```
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}
function throttle(fn, limit) {
  let waiting = false;
  return function (...args) {
    if (waiting) return;
    fn.apply(this, args);
    waiting = true;
    setTimeout(() => (waiting = false), limit);
  };
}
```

Real-time example: Debounce: search-as-you-type on an e-commerce site (call the API 400 ms after the last keystroke). Throttle: scroll and resize handlers, infinite scroll, protection from button spamming.

Q8. Shallow copy vs deep copy? [Intermediate]

Spread and `Object.assign` copy only the first level; nested objects are still shared by reference. `structuredClone(obj)` makes a real deep copy. `JSON.parse(JSON.stringify(obj))` is a hack that loses Dates, `undefined`, functions and Maps.

```
const a = { n: { x: 1 } };
const b = { ...a };
b.n.x = 99;
console.log(a.n.x);      // 99 -> nested object is shared
const c = structuredClone(a);
c.n.x = 5;               // a is not affected
```

Real-time example: React relies on immutable updates. `state.items.push(x)` does not trigger a re-render because the reference is unchanged; use `setItems([...items, x])`.

Q9. Explain map, filter and reduce with an example. [Basic]

`map` transforms every element and returns a new array of the same length. `filter` keeps the elements that pass a test. `reduce` folds an array into a single value. None of them mutate the original array.

```
const cart = [
  { name: 'Shoes', price: 2000, qty: 1, inStock: true },
  { name: 'Shirt', price: 800,  qty: 3, inStock: true },
  { name: 'Cap',   price: 300,  qty: 1, inStock: false },
];
const available = cart.filter(i => i.inStock);
const names = available.map(i => i.name);
const total = available.reduce((sum, i) => sum + i.price * i.qty, 0);
console.log(names, total); // ['Shoes', 'Shirt'] 4400
```

Real-time example: Cart totals, order summaries and dashboard statistics are typical `reduce` use cases.

Q10. What are event bubbling, capturing and event delegation? [Intermediate]

An event travels down from the root to the target (capturing) and then bubbles back up. With **delegation** you attach one listener on a parent and check `event.target`, instead of attaching listeners to every child. `stopPropagation()` stops the travel; `preventDefault()` stops the browser default action. React itself attaches its listeners at the root container and delegates from there.

Real-time example: A product grid with 1,000 'Add to cart' buttons needs one listener on the grid that reads `event.target.dataset.id`, not 1,000 listeners.

2. React Core and Hooks

This is the heart of the interview. For 2-3 years of experience, interviewers expect you to explain why, not only how, and to connect answers to projects you built.

Q11. What is the virtual DOM and how does reconciliation work? [Basic]

The virtual DOM is a lightweight JavaScript representation of the UI. When state changes, React builds a new virtual tree, diffs it against the previous one (**reconciliation**) and applies only the minimal changes to the real DOM. Two heuristics keep the diff fast: elements of different types produce different trees, and `key` identifies stable children in lists. Since React 16 the Fiber architecture splits rendering into small units of work that can be paused and prioritised.

Real-time example: When one product's quantity changes in a cart, React updates only that text node, not the whole list. Note: the virtual DOM is not magically faster than hand-written DOM code; its benefit is the declarative programming model.

Q12. Props vs state? What is 'lifting state up'? [Basic]

- **Props** are read-only inputs passed from parent to child.
- **State** is data owned by a component; changing it triggers a re-render.
- **Lifting state up** means moving state to the closest common parent so that sibling components can share it.

Real-time example: A `ProductCard` receives `product` as props, while the quantity selector keeps `qty` in its own state. When the header cart badge also needs the count, the cart state is lifted to a common parent or placed in a store.

Q13. Why do we need keys in lists? Why is the index a bad key? [Basic]

Keys let React match old and new children during reconciliation. Index keys break as soon as the list is reordered, filtered or has items removed, because React reuses the wrong component instances.

```
// Bad: index as key
{items.map((item, i) => <Row key={i} item={item} />)}
// Good: stable unique id
{items.map(item => <Row key={item.id} item={item} />)}
```

Real-time example: If the first cart item is deleted with index keys, input values or checkbox state can jump to the wrong row.

Q14. What are the Rules of Hooks and why do they exist? [Basic]

Call hooks only at the top level (never inside loops, conditions or nested functions) and only inside function components or custom hooks. React tracks hooks by their call order on every render, so a changing order corrupts state. The `eslint-plugin-react-hooks` rules enforce this.

Q15. Explain useEffect: dependency array, cleanup and common mistakes. [Intermediate]

- Runs after the browser paints.
- `[]` runs once after mount; no array runs after every render; `[a, b]` runs when `a` or `b` change.
- The returned cleanup function runs before the next effect and on unmount.
- Common mistakes: missing dependencies (stale data), objects or functions recreated every render as dependencies (infinite loops), and forgetting cleanup (memory leaks).

```
useEffect(() => {
  const controller = new AbortController();
  async function load() {
    try {
      const res = await fetch(`/api/orders?user=${userId}`,
                              { signal: controller.signal });
      if (!res.ok) throw new Error('Request failed');
      setOrders(await res.json());
```

```
    } catch (e) {
      if (e.name !== 'AbortError') setError(e.message);
    }
  }
  load();
  return () => controller.abort(); // cleanup prevents race condition
}, [userId]);
```

Real-time example: Race condition: a user switches quickly between two profiles and the slower first response arrives last, overwriting the correct data. Abort (or an ignore flag) fixes it. In production, libraries such as React Query or SWR are preferred over hand-written fetch effects. In development, StrictMode runs effects twice on purpose to expose missing cleanups.

Q16. Is setState synchronous? Why use the functional update form? [Intermediate]

State updates are batched and the `state` value inside a render is a snapshot. React 18 batches automatically, even inside promises and timeouts. Use the functional form when the new value depends on the previous one.

```
setCount(count + 1);
setCount(count + 1);    // total change: +1 (both use the same stale count)
setCount(c => c + 1);
setCount(c => c + 1);   // total change: +2
```

Real-time example: A 'quantity +' button clicked quickly, or a counter updated inside `setInterval`, should always use `setQty(q => q + 1)`.

Q17. useMemo vs useCallback vs React.memo? When should you NOT use them? [Intermediate]

`React.memo` skips re-rendering a component when its props are shallow-equal. `useCallback` memoises a function reference; `useMemo` memoises a computed value. They work together: an inline function passed to a memoised child defeats `memo`. They have their own cost, so measure with the Profiler first and do not wrap everything.

```
const ProductCard = React.memo(function ProductCard({ product, onAdd }) {
  return <button onClick={() => onAdd(product.id)}>{product.name}</button>;
});

function Catalog({ products, query }) {
  const visible = useMemo(
    () => products.filter(p =>
      p.name.toLowerCase().includes(query.toLowerCase())),
    [products, query]
  );
  const handleAdd = useCallback((id) => dispatch({ type: 'ADD', id }), []);
  return visible.map(p => (
    <ProductCard key={p.id} product={p} onAdd={handleAdd} />
  ));
}
```

Real-time example: Typing in a filter box that re-renders 5,000 product cards on every keystroke is a classic case. Memoised cards plus stable callbacks make only changed cards re-render.

Q18. What is useRef and when do you use it? [Intermediate]

`useRef` returns a persistent mutable container (`ref.current`). Changing it does not trigger a re-render. Uses: accessing DOM nodes, storing timer or socket IDs, keeping the previous value, and avoiding stale closures.

```
const inputRef = useRef(null);
const timerRef = useRef(null);
useEffect(() => { inputRef.current.focus(); }, []);
```

Real-time example: In a real-time interview app, a WebSocket or MediaRecorder instance is stored in a ref so that re-renders do not recreate the connection.

Q19. Context API vs Redux vs Zustand vs React Query? [Intermediate]

Context removes prop drilling, but every consumer re-renders when the value changes, so use it for low-frequency data (theme, logged-in user, locale). Redux Toolkit or Zustand suit complex, frequently changing client state and give devtools, middleware and selectors. **Server state** (API data) belongs in React Query or SWR, which handle caching, refetching and loading states.

Real-time example: A common split: auth user and theme in Context, shopping cart in Redux Toolkit or Zustand, product and order lists in React Query.

Q20. When would you choose useReducer? Show a cart reducer. [Intermediate]

Use `useReducer` when the next state depends on the previous state, when state has several related fields, or when transitions are complex. The reducer is a pure function, which makes it easy to test.

```
function cartReducer(state, action) {
  switch (action.type) {
    case 'ADD': {
      const found = state.find(i => i.id === action.item.id);
      return found
        ? state.map(i => i.id === found.id ? { ...i, qty: i.qty + 1 } : i)
        : [...state, { ...action.item, qty: 1 }];
    }
    case 'REMOVE':
      return state.filter(i => i.id !== action.id);
    default:
      return state;
  }
}
const [cart, dispatch] = useReducer(cartReducer, []);
dispatch({ type: 'ADD', item: product });
```

Q21. Controlled vs uncontrolled components? How do you handle forms? [Basic]

In a controlled component React state is the single source of truth for the input value. In an uncontrolled component the DOM keeps the value and you read it through a ref. For large forms, React Hook Form (uncontrolled under the hood, fewer re-renders) with Zod or Yup validation is the usual choice.

```
const [email, setEmail] = useState('');
<input value={email} onChange={e => setEmail(e.target.value)} />  // controlled
<input ref={fileRef} type="file" />                               // uncontrolled
```

Real-time example: Login and checkout forms need validation, error messages and disabled submit buttons, which is why they are usually controlled or handled by React Hook Form. File inputs are always uncontrolled.

Q22. What is a custom hook? Write useDebounce. [Intermediate]

A custom hook is a function whose name starts with `use` and that composes other hooks to reuse stateful logic. Each component that calls it gets its own independent state; the hook shares logic, not state.

```
function useDebounce(value, delay = 400) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);
  }, [value, delay]);
  return debounced;
}
// usage
const debouncedQuery = useDebounce(query, 400);
useEffect(() => {
  if (debouncedQuery) searchProducts(debouncedQuery);
}, [debouncedQuery]);
```

Real-time example: Other hooks teams write: `useFetch`, `useLocalStorage`, `useAuth`, `useOnlineStatus`, `useWebSocket`.

Q23. How do you optimise the performance of a React application? [Advanced]

- **Measure first:** React DevTools Profiler, Lighthouse and Web Vitals (LCP, INP, CLS).
- **Avoid needless renders:** `React.memo`, stable props, move state down, split big components.
- **Code splitting:** `React.lazy` + `Suspense`, route-based chunks.
- **Virtualise long lists:** react-window or react-virtuoso.
- **Images:** compress, use WebP or AVIF, `loading='lazy'`, set width and height, serve from a CDN.
- **Network:** debounce inputs, paginate APIs, cache responses.
- **Bundle:** tree shaking, analyse with webpack-bundle-analyzer.

```
const Dashboard = React.lazy(() => import('./pages/Dashboard'));

<Suspense fallback={<Spinner />}>
  <Dashboard />
</Suspense>
```

Real-time example: Interviewers often ask: 'You mention improving load time by 30%. How?' A solid answer names the tool and the steps: measured LCP before and after in Lighthouse, compressed and lazy-loaded product images, split the bundle, and removed unnecessary re-renders. Quote only numbers you can explain and reproduce.

Q24. What are Error Boundaries? [Intermediate]

An error boundary is a class component that catches rendering errors in its child tree and shows a fallback UI instead of crashing the whole app. It does not catch errors in event handlers, async code or server-side rendering. The `react-error-boundary` library gives a hook-friendly version.

```
class ErrorBoundary extends React.Component {
  state = { hasError: false };
  static getDerivedStateFromError() { return { hasError: true }; }
  componentDidCatch(error, info) { logToServer(error, info.componentStack); }
  render() {
    return this.state.hasError
      ? <h2>Something went wrong.</h2>
      : this.props.children;
  }
}
```

Real-time example: Wrap each dashboard widget separately so one failing chart does not blank the whole page.

Q25. What is new in React 18? [Intermediate]

- **Automatic batching** of updates everywhere (promises, timeouts, native events).
- **Concurrent rendering:** React can interrupt and prioritise renders.
- `useTransition` and `startTransition` mark updates as non-urgent; `useDeferredValue` defers a value.
- **Suspense** for data fetching and streaming server rendering.
- `createRoot` replaces `ReactDOM.render`; `useId` generates stable IDs.

```
const [isPending, startTransition] = useTransition();
function onChange(e) {
  setText(e.target.value);                      // urgent: keep typing smooth
  startTransition(() => setFilter(e.target.value)); // non-urgent: heavy list
}
```

Real-time example: Typing in a search box stays responsive while a large filtered list updates in the background.

Q26. How do you implement protected routes with role-based access? [Intermediate]

Wrap routes in a guard component that checks authentication and role. This is only a UX guard: the real security must be enforced by the API with token verification and role checks on the server.

```
function ProtectedRoute({ children, roles }) {
  const { user, loading } = useAuth();
  if (loading) return <Spinner />;
  if (!user) return <Navigate to="/login" replace />;
  if (roles && !roles.includes(user.role)) return <Navigate to="/403" replace />;
  return children;
}
// <Route path="/admin" element={
//   <ProtectedRoute roles={['admin']}><Admin /></ProtectedRoute>} />
```

Real-time example: In a recruitment app, candidates see only their own interview, recruiters see assigned candidates and admins manage everything.

Q27. How do you handle an expired access token in React (axios interceptor)? [Advanced]

Keep the access token short-lived (about 15 minutes) and the refresh token long-lived in an httpOnly cookie. An interceptor catches 401, calls the refresh endpoint once, then replays the failed request. Queue concurrent 401 responses so only one refresh call is made.

```
api.interceptors.response.use(
  (res) => res,
  async (error) => {
    const original = error.config;
    if (error.response?.status === 401 && !original._retry) {
      original._retry = true;
      await api.post('/auth/refresh');   // refresh token is an httpOnly cookie
      return api(original);              // replay the failed request
    }
    return Promise.reject(error);
  }
);
```

Q28. CSR vs SSR vs SSG vs ISR? Why Next.js? [Intermediate]

- **CSR:** the browser builds the page with JavaScript; simple, but slower first paint and weaker SEO.
- **SSR:** HTML is generated per request on the server; good SEO and fresh data.
- **SSG:** HTML is generated at build time; fastest, best for static content.
- **ISR:** static pages regenerated in the background at intervals.

Next.js provides all four, plus file-based routing, API routes, image optimisation and the App Router with Server Components.

Real-time example: Product pages and blogs benefit from SSG/ISR for SEO and speed; a logged-in dashboard works fine as CSR.

Q29. Map class component lifecycle to hooks. useEffect vs useLayoutEffect? [Basic]

`componentDidMount` is `useEffect(fn, [])`; `componentDidUpdate` is `useEffect(fn, [deps])`; `componentWillUnmount` is the cleanup returned from `useEffect`. `useLayoutEffect` runs synchronously after DOM updates but before paint, so use it only when you must measure or adjust layout without flicker; otherwise use `useEffect`.

3. TypeScript

TypeScript is now expected for most React roles. Questions are practical: typing props, API responses and reusable utilities.

Q30. interface vs type? [Basic]

Both describe shapes. `interface` supports declaration merging and `extends`, and suits object contracts. `type` can also express unions, intersections, tuples, primitives, mapped and conditional types. Pick one convention per team.

```
interface User { id: number; name: string; email: string }
type Role = 'admin' | 'recruiter' | 'candidate';   // union: only `type`
type Admin = User & { role: 'admin'; permissions: string[] };
```

Q31. any vs unknown vs never? [Intermediate]

`any` switches off type checking. `unknown` is the safe alternative: you must narrow it before use. `never` represents a value that can never occur, used for exhaustive checks and functions that always throw.

```
function parse(json: string): unknown { return JSON.parse(json); }
const data = parse('{"id":1}');
if (typeof data === 'object' && data !== null && 'id' in data) {
  // safe to use data.id here
}
function assertNever(x: never): never { throw new Error('Unhandled: ' + x); }
```

Real-time example: A response from an external API is typed as `unknown` and validated with Zod before the app trusts it.

Q32. What are generics? Give an API example. [Intermediate]

Generics let you write reusable code that keeps type information. The type is supplied by the caller instead of being hard-coded.

```
interface ApiResponse<T> { data: T; error?: string; total?: number }

async function get<T>(url: string): Promise<ApiResponse<T>> {
  const res = await fetch(url);
  return res.json();
}
const products = await get<Product[]>('/api/products');
```

Q33. Name useful utility types. [Intermediate]

`Partial<T>` (all optional), `Required<T>`, `Pick<T, K>`, `Omit<T, K>`, `Record<K, V>`, `Readonly<T>`, `ReturnType<F>`.

```
type UpdateUserDto = Partial<Pick<User, 'name' | 'email'>>;
type PublicUser = Omit<User, 'passwordHash'>;
type Score = Record<'technical' | 'communication' | 'leadership', number>;
```

Real-time example: A PATCH endpoint accepts `Partial<...>`; a response type removes secret fields with `Omit`; an evaluation result is a `Record` of score categories.

Q34. How do you type props, state, events and refs in React with TypeScript? [Basic]

Define a props type, give `useState` a generic when the initial value does not show the full type, and use React's built-in event types.

```
type ButtonProps = {
```

```
  label: string;
  onClick: () => void;
  variant?: 'primary' | 'ghost';
};
function Button({ label, onClick, variant = 'primary' }: ButtonProps) {
  return <button className={variant} onClick={onClick}>{label}</button>;
}
const [user, setUser] = useState<User | null>(null);
const onChange = (e: React.ChangeEvent<HTMLInputElement>) =>
  setName(e.target.value);
const ref = useRef<HTMLInputElement>(null);
```

4. Node.js, Express and APIs

Full stack roles always include backend questions: authentication, REST design, security and error handling.

Q35. How does Node.js handle many requests with a single thread? [Intermediate]

Node uses an event-driven, non-blocking I/O model. The libuv event loop (timers, poll, check and close phases) hands I/O work such as files, DNS and crypto to the OS or a thread pool and continues with other requests. CPU-heavy work blocks the loop, so move it to `worker_threads`, a cluster, or a background queue.

Real-time example: Generating PDFs or resizing images inside a request handler stalls every user. Push such jobs to a queue (for example BullMQ with Redis) processed by a worker.

Q36. What is middleware in Express? Show auth and error middleware. [Basic]

Middleware are functions with access to `req`, `res` and `next()`. They run in order, and a middleware with four arguments `(err, req, res, next)` is an error handler.

```
const express = require('express');
const jwt = require('jsonwebtoken');
const app = express();
app.use(express.json());

app.use((req, res, next) => {            // logger
  console.log(req.method, req.url);
  next();
});

function auth(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ message: 'No token' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    res.status(401).json({ message: 'Invalid or expired token' });
  }
}
app.get('/api/orders', auth, getOrders);

app.use((err, req, res, next) => {       // global error handler
  console.error(err);
  res.status(err.status || 500).json({ message: err.message });
});
```

Q37. Explain JWT authentication end to end. Where should the token be stored? [Advanced]

- User logs in; the server compares the password with a **bcrypt** hash.
- The server issues a short-lived access token (about 15 minutes) and a longer refresh token (about 7 days).
- Tokens travel in `httpOnly`, `Secure`, `SameSite` cookies, or the access token in the `Authorization` header.
- Middleware verifies the signature and expiry on every request.
- On 401 the client calls `/auth/refresh` to get a new access token.
- A JWT is signed, not encrypted: never put secrets inside it. For logout or revocation use short expiry, refresh-token rotation and a denylist in Redis.

Storage trade-off: `localStorage` is exposed to XSS; httpOnly cookies are safe from JavaScript but need CSRF protection (SameSite, CSRF token).

```
const ok = await bcrypt.compare(password, user.passwordHash);
if (!ok) return res.status(401).json({ message: 'Invalid credentials' });

const access = jwt.sign(
  { sub: user.id, role: user.role },
  process.env.JWT_SECRET,
  { expiresIn: '15m' }
```

```
);
res.cookie('token', access, { httpOnly: true, secure: true, sameSite: 'strict' });
res.json({ id: user.id, role: user.role });
```

Q38. REST API design: HTTP methods, status codes and idempotency. [Intermediate]

Use nouns for resources (`GET /api/orders/42`), verbs come from the method. `GET`, `PUT` and `DELETE` are idempotent (repeating them gives the same result); `POST` is not.

- **2xx:** 200 OK, 201 Created, 204 No Content
- **4xx:** 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Validation error, 429 Too Many Requests
- **5xx:** 500 Internal Server Error, 502/503 upstream or overload

Version the API (`/v1/`), paginate lists and return consistent error bodies.

Real-time example: Creating an order with `POST /api/orders` returns 201 and the new order; a duplicate client retry must not create a second order (see idempotency key in the project section).

Q39. How do you handle errors in async Express routes? [Intermediate]

Express 4 does not catch rejected promises from async handlers, so a wrapper forwards the error to the error middleware with `next(err)`. Express 5 handles this natively.

```
const asyncHandler = (fn) => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);

app.get('/api/products/:id', asyncHandler(async (req, res) => {
  const product = await Product.findById(req.params.id);
  if (!product) return res.status(404).json({ message: 'Not found' });
  res.json(product);
}));
```

Q40. How do you secure a Node/Express API? [Advanced]

Use `helmet` for security headers, a CORS whitelist, rate limiting, input validation (Joi or Zod), parameterised queries against SQL injection, output escaping against XSS, bcrypt for passwords, HTTPS everywhere, secrets in environment variables, `npm audit` for dependencies, and CSRF protection when cookies are used.

```
const helmet = require('helmet');
const cors = require('cors');
const rateLimit = require('express-rate-limit');

app.use(helmet());
app.use(cors({ origin: ['https://myshop.com'], credentials: true }));
app.use('/api/auth', rateLimit({ windowMs: 15 * 60 * 1000, max: 20 }));

// parameterised query prevents SQL injection
db.query('SELECT * FROM users WHERE email = ?', [email]);
```

Real-time example: Rate limiting on the login route blocks brute-force attempts; validation rejects malformed payloads before they reach the database.

Q41. Implement pagination. Offset vs cursor? [Intermediate]

Offset (`LIMIT/OFFSET`) is simple but gets slower on large offsets and can skip or repeat rows when data changes. Cursor pagination (`WHERE id > lastId ORDER BY id LIMIT n`) is faster and stable, which suits infinite scroll.

```
app.get('/api/products', async (req, res) => {
  const page = Math.max(parseInt(req.query.page) || 1, 1);
  const limit = Math.min(parseInt(req.query.limit) || 20, 100);
  const [rows] = await db.query(
    'SELECT id, name, price FROM products ORDER BY id LIMIT ? OFFSET ?',
    [limit, (page - 1) * limit]
  );
```

```
  res.json({ page, limit, data: rows });
});
```

Q42. Authentication vs authorization? Implement role-based access control. [Intermediate]

Authentication answers 'who are you?' (failure gives 401). Authorization answers 'what may you do?' (failure gives 403).

```
const authorize = (...roles) => (req, res, next) =>
  roles.includes(req.user.role)
    ? next()
    : res.status(403).json({ message: 'Forbidden' });

app.delete('/api/candidates/:id', auth, authorize('admin'), removeCandidate);
```

Real-time example: In a hiring platform: candidates read only their own data, recruiters see assigned candidates, admins manage users, and every sensitive action is written to an audit log.

Q43. Why FastAPI? How does it compare with Express? [Intermediate]

FastAPI is an async Python framework built on type hints. It gives Pydantic validation, automatic OpenAPI docs at `/docs`, dependency injection with `Depends`, and runs on an ASGI server (Uvicorn). It is a natural fit when the backend calls Python AI libraries. Express is minimal and flexible but needs extra libraries for validation and docs.

```
from fastapi import FastAPI, Depends, HTTPException
from pydantic import BaseModel

app = FastAPI()

class CandidateIn(BaseModel):
    name: str
    email: str
    role: str

@app.post('/candidates', status_code=201)
async def create_candidate(body: CandidateIn, user=Depends(get_current_user)):
    if user.role != 'recruiter':
        raise HTTPException(status_code=403, detail='Forbidden')
    return await save_candidate(body)
```

Q44. WebSockets vs HTTP polling vs Server-Sent Events? [Intermediate]

- **Polling:** the client asks repeatedly; simple but wasteful and delayed.
- **SSE:** one-way server-to-client stream over HTTP; good for notifications and live feeds.
- **WebSocket:** one persistent, full-duplex connection; best for chat, live transcripts and collaboration.

Production WebSocket clients need heartbeats, authentication on connect and reconnection with exponential backoff.

```
const ws = new WebSocket('wss://api.example.com/interview/123');
ws.onmessage = (e) => addTranscript(JSON.parse(e.data));
ws.onclose = () => {
  // exponential backoff: 1s, 2s, 4s ... capped
  setTimeout(reconnect, Math.min(1000 * 2 ** attempt, 30000));
};
```

Real-time example: A live interview or chat screen uses WebSocket for transcript and messages, while an order-status page can use SSE or occasional polling.

5. Databases: SQL, MongoDB and Redis

SQL queries are common in every MNC technical round. Practise writing them by hand.

Q45. SQL vs NoSQL: how do you choose? [Basic]

SQL (MySQL, PostgreSQL) has fixed schemas, joins and ACID transactions, ideal for orders, payments and anything relational. NoSQL (MongoDB) stores flexible documents, scales horizontally and suits evolving or nested data such as catalogues, logs or chat messages. Many systems use both.

Real-time example: Orders and payments in MySQL or PostgreSQL for consistency; product catalogue or interview transcripts in MongoDB; sessions and cache in Redis.

Q46. Explain joins with an e-commerce example. [Basic]

`INNER JOIN` returns matching rows only. `LEFT JOIN` returns all rows from the left table plus matches (NULL when none). `RIGHT JOIN` is the mirror. `FULL JOIN` returns everything from both sides.

```
-- only users who have paid orders
SELECT u.name, o.id AS order_id, o.total
FROM users u
INNER JOIN orders o ON o.user_id = u.id
WHERE o.status = 'PAID';

-- all users, even those with zero orders
SELECT u.name, COUNT(o.id) AS order_count
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
GROUP BY u.id, u.name;
```

Q47. What are indexes? When do they hurt? [Intermediate]

An index (usually a B-tree) lets the database find rows without scanning the whole table. It speeds up reads and sorting but slows writes and uses storage. A composite index follows the leftmost-prefix rule. Indexes help little on low-cardinality columns, and wrapping a column in a function in the `WHERE` clause (`WHERE YEAR(created_at) = 2026`) prevents index use. Check plans with `EXPLAIN`.

```
CREATE INDEX idx_orders_user_status ON orders (user_id, status);
EXPLAIN SELECT * FROM orders WHERE user_id = 42 AND status = 'PAID';
```

Real-time example: A slow 'my orders' page is often fixed by indexing `orders.user_id`.

Q48. Explain ACID and write an order-placement transaction. [Advanced]

Atomicity (all or nothing), **Consistency** (valid state before and after), **Isolation** (concurrent transactions do not see partial work) and **Durability** (committed data survives crashes).

```
START TRANSACTION;
INSERT INTO orders (user_id, total) VALUES (42, 4400);
SET @order_id = LAST_INSERT_ID();
UPDATE products SET stock = stock - 1 WHERE id = 7 AND stock > 0;
-- if affected rows = 0 -> ROLLBACK (out of stock)
INSERT INTO payments (order_id, amount, status)
VALUES (@order_id, 4400, 'PENDING');
COMMIT;
```

Real-time example: The `stock > 0` condition inside the UPDATE prevents overselling when two customers buy the last item at the same time.

Q49. Common SQL coding questions asked in interviews [Intermediate]

Practise these four patterns: second highest value, duplicates, top-N per aggregate, and rows above a group average. Also remember: `WHERE` filters rows before grouping, `HAVING` filters groups after aggregation.

```
-- 2nd highest salary
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- duplicate emails
SELECT email, COUNT(*) AS c FROM users GROUP BY email HAVING c > 1;

-- top 3 products by revenue
SELECT product_id, SUM(price * qty) AS revenue
FROM order_items GROUP BY product_id ORDER BY revenue DESC LIMIT 3;

-- employees earning above their department average
SELECT e.* FROM employees e
JOIN (SELECT dept_id, AVG(salary) AS avg_sal FROM employees GROUP BY dept_id) d
  ON e.dept_id = d.dept_id
WHERE e.salary > d.avg_sal;
```

Q50. What is the N+1 query problem? [Intermediate]

Loading a list with one query and then running one extra query per row (N more). It kills performance as the list grows. Fix it with a `JOIN`, a single `WHERE id IN (...)` query, eager loading or `populate`, or batching with DataLoader.

Real-time example: Fetching 20 orders and then querying items separately for each order makes 21 queries; one join or one `IN` query makes 1 or 2.

Q51. MongoDB: embed vs reference? Write an aggregation. [Intermediate]

Embed data that is read together and has a bounded size (an order's line items). Reference data that is shared, large or grows without limit (users, products). The aggregation pipeline processes documents in stages.

```
db.orders.aggregate([
  { $match: { status: 'PAID' } },
  { $group: { _id: '$userId', total: { $sum: '$amount' }, count: { $sum: 1 } } },
  { $sort: { total: -1 } },
  { $limit: 5 }
]);   // top 5 customers by spend
```

Q52. How does Redis caching work? Explain cache-aside. [Intermediate]

In cache-aside the app checks Redis first; on a miss it reads the database, stores the result with a TTL and returns it. On update, delete or refresh the key. Beyond caching, Redis is used for sessions, rate-limit counters, pub/sub and job queues. Risks: stale data, and a cache stampede when a popular key expires (mitigate with jittered TTLs or locks).

```
async function getProduct(id) {
  const key = `product:${id}`;
  const cached = await redis.get(key);
  if (cached) return JSON.parse(cached);                  // cache hit
  const product = await db.findProduct(id);               // cache miss
  await redis.set(key, JSON.stringify(product), 'EX', 300); // TTL 5 min
  return product;
}
// on update: await redis.del(`product:${id}`)  (invalidate)
```

6. Docker, Git, CI/CD and AWS Basics

Juniors and mid-level developers are expected to know the basics of shipping code, not to be DevOps experts.

Q53. Docker: image vs container? Write a Dockerfile and a compose file. [Basic]

An **image** is a read-only template built in layers; a **container** is a running instance of an image. Copy `package.json` first so the dependency layer is cached when only source code changes. Use `.dockerignore`, never bake secrets into images, and use multi-stage builds to keep images small.

```
# Dockerfile
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 5000
CMD ["node", "server.js"]
```

```
# docker-compose.yml
services:
  api:
    build: ./backend
    ports: ['5000:5000']
    env_file: .env
    depends_on: [db, redis]
  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_PASSWORD}
    volumes: ['dbdata:/var/lib/mysql']
  redis:
    image: redis:7-alpine
volumes:
  dbdata:
```

Real-time example: Useful commands: `docker build -t app .`, `docker run -p 5000:5000 app`, `docker ps`, `docker logs <id>`, `docker compose up -d`. Docker fixes the 'works on my machine' problem because the same image runs in development, testing and production.

Q54. Git: merge vs rebase? How do you resolve conflicts? [Basic]

`merge` joins branches and keeps history with a merge commit. `rebase` replays your commits on top of another branch for a linear history; never rebase shared public branches. To resolve conflicts: pull latest, open conflicted files, keep the correct code between the markers, `git add`, then continue the merge or rebase. Also know `git stash`, `cherry-pick`, and `revert` (safe, new commit) versus `reset` (rewrites history).

Real-time example: Typical team flow: feature branch, small commits, pull request, code review, squash merge into main.

Q55. What is CI/CD? How would you set it up for a MERN app? [Intermediate]

Continuous Integration runs automated checks on every push; Continuous Delivery or Deployment ships passing builds. A basic pipeline (GitHub Actions) is: install, lint, test, build, build and push the Docker image, deploy. Keep configuration in environment variables or a secrets manager and never commit `.env` files.

Real-time example: A pull request triggers lint and unit tests; a merge to `main` builds a Docker image and deploys it to the server.

Q56. Which AWS services should a full stack developer know? [Basic]

- **EC2:** virtual servers for the API.
- **S3:** object storage for images, build files and recordings (enable encryption and signed URLs).
- **RDS:** managed MySQL or PostgreSQL.

- **IAM:** users and roles with least privilege.
- **CloudFront:** CDN in front of S3 or the app.
- **Lambda:** serverless functions.

Real-time example: A common beginner deployment: React build on S3 + CloudFront, API in Docker on EC2, database on RDS.

7. Python and AI Basics (HR-Agent Project)

Because your resume lists an LLM-based project, expect questions on how you integrated AI, how you handled failures, and how you protected candidate data. Answer only about parts you actually built.

Q57. How did you integrate an LLM into your application? [Advanced]

- Wrap the LLM in a service layer so the provider can be changed.
- Build the prompt from a **system prompt** (role and rules), the question bank or rubric, and the conversation history.
- **Stream** tokens to the UI to hide latency.
- Control cost with token limits and by summarising old history.
- Reduce hallucination by grounding in the rubric, using low temperature for scoring and asking for **structured JSON** that is validated (for example with Pydantic) before saving.
- Add timeouts, retries with backoff, rate-limit handling and logging.
- Review sample outputs by hand to evaluate quality.

```
import json

async def evaluate(transcript: str) -> dict:
    prompt = (
        'You are an interview evaluator. Score 1-10 for technical, '
        'communication, behavioral, leadership and confidence. '
        'Return ONLY valid JSON.\n\nTranscript:\n' + transcript
    )
    raw = await llm.complete(prompt, temperature=0.2)  # llm = your client wrapper
    data = json.loads(raw)       # validate with a Pydantic model before saving
    return data
```

Q58. Prompt engineering basics: what makes a good prompt? [Basic]

State the role, give context, add constraints, define the exact output format and include one or two examples (few-shot). Keep system instructions separate from user input. Use low temperature (0 to 0.3) for scoring and consistency, and a higher value for varied question generation.

Real-time example: Interviewer profiles (Technical Screener, Behavioral Interviewer and so on) can be implemented as different system prompts, with difficulty, tone and scoring criteria passed as configuration.

Q59. Describe the flow of a real-time AI voice interview. [Advanced]

Microphone audio is captured in the browser and streamed in small chunks over WebSocket or WebRTC. Speech-to-text produces partial and final transcripts, which go to the LLM with the history. The reply is streamed back and converted to speech for playback. Each message carries a sequence number so the transcript stays in order and can be re-synchronised after a disconnect. Latency is the main challenge: stream every stage and avoid waiting for complete results.

Real-time example: If the connection drops, the client reconnects and sends the last sequence number it received; the server replays only the missing messages.

Q60. Python basics frequently asked in full stack + AI roles [Basic]

- List (mutable) vs tuple (immutable); dict and set for fast lookup.
- List comprehension, generators (`yield`) and decorators.
- The GIL: threads do not run Python bytecode in parallel, so use `async` for I/O-bound work and multiprocessing for CPU-bound work.
- Virtual environments (`venv`) and `requirements.txt`.
- Type hints, exceptions with `try/except/finally`, context managers (`with`).

```
squares = [x * x for x in range(5) if x % 2 == 0]   # [0, 4, 16]
```

```
import time
def timer(fn):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = fn(*args, **kwargs)
        print(fn.__name__, round(time.time() - start, 3), 's')
        return result
    return wrapper

@timer
def slow():
    time.sleep(0.2)
```

Q61. What is RAG (Retrieval-Augmented Generation)? [Intermediate]

RAG retrieves relevant documents (using embeddings and a vector search) and adds them to the prompt so the LLM answers from your data instead of guessing. Steps: split documents into chunks, create embeddings, store them in a vector database, retrieve the top matches for a query, and include them as context.

Real-time example: A recruiter chatbot that answers questions from company policy documents, or a system that picks interview questions from a large question bank by semantic similarity.

Q62. How do you protect candidate privacy in an interview and proctoring system? [Intermediate]

Take explicit consent before recording, encrypt data in transit (TLS) and at rest (for example AES-256 or S3 server-side encryption), use role-based access, signed expiring URLs for recordings, audit logs for every access, and a defined retention and deletion policy. Treat proctoring flags as signals for a human to review, not as automatic rejection, because detection can produce false positives and bias.

8. Project and Scenario Questions

Most MNC technical rounds spend 30 to 40 percent of the time on your own project. Know every number and every technology on your resume.

Q63. Walk me through your project. [Basic]

Use the structure **Problem, Solution, Tech stack, Your role, Challenge, Result**. Keep it to 60 to 90 seconds, then let the interviewer dig deeper.

Sample (based on your resume): 'I worked on an e-commerce web application at Clahan Technologies, mainly on the React front end. I built reusable components for product listing, filters, cart and the user dashboard, managed state with React Hooks and integrated REST APIs for products, users and orders. I also built Express.js endpoints on the backend. One challenge was page load time, which I improved by optimising images and component rendering.'

Q64. How do you defend numbers like '30% faster' or '10,000 API calls per month'? [Advanced]

Be ready to explain what was measured, with which tool, and how: Lighthouse or Chrome DevTools for page load, Postman or logs for response times, and server logs or monitoring for request counts. If you cannot explain the method, reword the claim honestly rather than defend it. Interviewers value accurate answers over big numbers.

Real-time example: Good answer: 'Average API response for the order list went from X ms to Y ms after adding an index and pagination; I measured it in Postman and server logs.'

Q65. Design a checkout flow. How do you avoid duplicate orders and overselling? [Advanced]

- The client sends an **idempotency key** with the order request; the server returns the existing order if the key was already used.
- Reduce stock atomically inside a database transaction (`stock = stock - 1 WHERE stock > 0`).
- Order moves through states: PENDING, PAID, SHIPPED, CANCELLED.
- Confirm payment with the provider's **webhook**, not only the browser redirect.
- Disable the Pay button after the first click.

```
app.post('/api/orders', auth, async (req, res) => {
  const key = req.headers['idempotency-key'];
  const existing = await Order.findOne({ idempotencyKey: key });
  if (existing) return res.status(200).json(existing);  // duplicate click or retry
  const order = await Order.create({
    ...req.body, idempotencyKey: key, status: 'PENDING',
  });
  res.status(201).json(order);
});
```

Q66. The application is slow. How do you find the cause? [Advanced]

Work from the user inwards: 1) Browser: DevTools Performance and Network tabs, Lighthouse, large bundles or images, re-renders. 2) Network: payload size, number of requests, caching headers, compression. 3) API: slow endpoints in logs or APM, blocking code. 4) Database: slow query log, `EXPLAIN`, missing indexes, N+1 queries. 5) Infrastructure: CPU and memory, connection limits. Fix the biggest bottleneck first and measure again.

Q67. How would you scale the system to 10 times more traffic? [Advanced]

Keep the API stateless and run several instances behind a load balancer; cache with Redis and a CDN; add database indexes and read replicas; paginate; move heavy work to background queues; apply rate limiting; add monitoring and alerts; and load-test before launch.

Q68. How do you detect tab switching and other proctoring violations in the browser?

[Intermediate]

The Page Visibility API (`visibilitychange`) and the window `blur` event detect tab or window switches. Camera checks (face present, multiple faces, head direction) use a computer-vision model running in the browser or on the server; name the library you actually used. Send each violation with a timestamp to the backend and store it for review. Always get consent and expect false positives.

```
document.addEventListener('visibilitychange', () => {
  if (document.hidden) logViolation('TAB_SWITCH', Date.now());
});
window.addEventListener('blur', () => logViolation('WINDOW_BLUR', Date.now()));
```

9. Coding Round and Machine Coding

Service MNCs usually ask one or two simple logic problems. Product companies add medium DSA, polyfills and React machine-coding tasks. Explain your approach and complexity before typing.

Q69. Two Sum [Basic]

Use a hash map of value to index. For each number look for `target - number`. Time O(n), space O(n).

```
function twoSum(nums, target) {
  const seen = new Map();
  for (let i = 0; i < nums.length; i++) {
    const need = target - nums[i];
    if (seen.has(need)) return [seen.get(need), i];
    seen.set(nums[i], i);
  }
  return [];
}
```

Q70. Palindrome check, reverse a string, remove duplicates, count frequency [Basic]

These four are the most common warm-up questions.

```
const isPalindrome = (s) => {
  const t = s.toLowerCase().replace(/[^a-z0-9]/g, '');
  return t === [...t].reverse().join('');
};
const unique = [...new Set(arr)];
const freq = arr.reduce((m, x) => {
  m[x] = (m[x] || 0) + 1;
  return m;
}, {});
```

Q71. Flatten a nested array [Intermediate]

Recursion with `reduce`. The built-in is `arr.flat(Infinity)`.

```
const flatten = (arr) =>
  arr.reduce(
    (acc, x) => (Array.isArray(x) ? acc.concat(flatten(x)) : acc.concat(x)),
    []
  );
flatten([1, [2, [3, [4]]]]); // [1, 2, 3, 4]
```

Q72. Group anagrams [Intermediate]

Sort the letters of each word to build a key, then group by key. Time O(n k log k).

```
function groupAnagrams(words) {
  const map = {};
  for (const w of words) {
    const key = [...w].sort().join('');
    (map[key] ||= []).push(w);
  }
  return Object.values(map);
}
groupAnagrams(['eat', 'tea', 'tan', 'nat']); // [['eat','tea'], ['tan','nat']]
```

Q73. Implement Promise.all (polyfill) [Advanced]

Product companies like polyfills because they test promises, closures and ordering. Results must keep the input order, and the first rejection rejects the whole promise.

```
function promiseAll(promises) {
```

```
  return new Promise((resolve, reject) => {
    const results = [];
    let done = 0;
    if (promises.length === 0) return resolve(results);
    promises.forEach((p, i) => {
      Promise.resolve(p)
        .then((val) => {
          results[i] = val;
          if (++done === promises.length) resolve(results);
        })
        .catch(reject);
    });
  });
}
```

Q74. React machine coding: search box with debounce and API call (autocomplete) [Advanced]

Requirements to mention before coding: debounce input, cancel stale requests, loading and empty states, error handling, keyboard support and accessibility. This uses the `useDebounce` hook shown earlier.

```
function Search() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);
  const [loading, setLoading] = useState(false);
  const debounced = useDebounce(query, 400);

  useEffect(() => {
    if (!debounced.trim()) { setResults([]); return; }
    const controller = new AbortController();
    setLoading(true);
    const url = `/api/products?search=${encodeURIComponent(debounced)}`;
    fetch(url, { signal: controller.signal })
      .then((r) => r.json())
      .then(setResults)
      .catch((e) => e.name !== 'AbortError' && console.error(e))
      .finally(() => { if (!controller.signal.aborted) setLoading(false); });
    return () => controller.abort();
  }, [debounced]);

  return (
    <div>
      <input value={query} onChange={(e) => setQuery(e.target.value)} />
      {loading && <p>Loading...</p>}
      <ul>{results.map((p) => <li key={p.id}>{p.name}</li>)}</ul>
    </div>
  );
}
```

Q75. React machine coding: infinite scroll [Advanced]

Use `IntersectionObserver` on the last item to trigger loading the next page instead of listening to scroll events. Also handle loading state, end of data and duplicate requests.

```
function useInfiniteScroll(loadMore, hasMore) {
  const observer = useRef();
  return useCallback((node) => {
    if (observer.current) observer.current.disconnect();
    observer.current = new IntersectionObserver(([entry]) => {
      if (entry.isIntersecting && hasMore) loadMore();
    });
    if (node) observer.current.observe(node);
  }, [loadMore, hasMore]);
}
// usage: <li ref={isLast ? lastRef : null}>...</li>
```

Q76. Output-based questions (quick-fire) [Basic]

- `typeof null` gives `'object'` (a historical bug).

- `typeof NaN` gives `'number'`.
- `[] + []` gives `''` (empty string).
- `0.1 + 0.2 === 0.3` gives `false` (floating-point precision).
- `[1, 2, 3].map(parseInt)` gives `[1, NaN, NaN]` because `map` passes the index as the radix.
- `1 < 2 < 3` is `true`, but `3 > 2 > 1` is `false` (`true > 1` is false).
- `[10, 9, 1].sort()` gives `[1, 10, 9]` because the default sort compares strings.

10. HR and Behavioural Round

Use the STAR method (Situation, Task, Action, Result) for experience questions, and keep every answer to about one minute.

Q77. Tell me about yourself. [Basic]

Use Present, Past, Future in 45 to 60 seconds.

Sample answer: 'I am Mounika, a Full Stack Developer with 2.4 years of experience at Clahan Technologies in Bangalore, working mainly with React, Node.js, Express and MySQL. I joined as an intern in July 2024 and moved into a full-time role, where I built reusable React components, integrated REST APIs and developed backend endpoints for an e-commerce platform. Recently I built an AI-powered HR Interview Agent using React, TypeScript, FastAPI and LLM services, which gave me experience with real-time communication, authentication and Docker. I am now looking for a role where I can grow in full stack development and AI-enabled products.'

Q78. Why do you want to leave your current company / why join us? [Basic]

Focus on growth, never on complaints. Mention what you learned, what you want next (larger scale, product ownership, newer technology) and one specific thing you know about the company's products or engineering.

Q79. What are your strengths and weaknesses? [Basic]

Pick strengths you can prove with an example (fast learner, ownership, clean reusable components). For a weakness choose a real but non-critical one and show what you are doing about it, for example: 'I used to take on too much without asking for help; now I raise blockers early and use daily check-ins.'

Q80. Describe a time you handled a tight deadline or a production bug. [Intermediate]

Answer in STAR form. **Situation:** a release was due and a bug appeared. **Task:** your responsibility. **Action:** how you reproduced, isolated and fixed it, what you communicated to the team. **Result:** outcome and what you changed afterwards (tests, monitoring, review checklist).

Q81. How do you handle disagreement with a teammate or reviewer? [Intermediate]

Listen first, discuss with data (benchmarks, documentation, a small prototype), focus on the product goal, escalate to the lead only if needed, and accept the team decision once made. Show that you can disagree without making it personal.

Q82. What is your expected salary? What is your notice period? [Basic]

Research the market range for your experience, role and city on AmbitionBox, Glassdoor or LinkedIn, then give a range rather than a single figure and say you are flexible for the right role. State your real notice period and any buy-out possibility honestly.

Q83. Do you have any questions for us? [Basic]

Always ask two or three: 'What does a typical sprint look like for this team?', 'What would success look like in the first 6 months?', 'How do you handle code reviews and testing?', 'What technologies is the team planning to adopt?'

11. Quick Revision: Cheat Sheets and Interview Tips

React hooks at a glance

Hook	Purpose	Typical use
`useState`	Local component state	Form inputs, toggles, counters
`useEffect`	Side effects after render	Data fetching, subscriptions, timers (with cleanup)
`useRef`	Mutable value or DOM node, no re-render	Focus input, store socket or timer id
`useMemo`	Cache a computed value	Filtering or sorting a large list
`useCallback`	Cache a function reference	Stable props for memoised children
`useContext`	Read context value	Theme, auth user
`useReducer`	Complex state transitions	Shopping cart, multi-step forms
`useTransition`	Mark updates as non-urgent	Keep typing smooth while a list filters
`useId`	Stable unique ids	Label and input pairing, SSR safe

What different MNC types usually test

Company type	Examples	Usual focus
Service MNCs	TCS, Infosys, Wipro, Cognizant, Accenture, Capgemini, HCLTech, Tech Mahindra	Aptitude or coding test, then JS/React fundamentals, REST, SQL queries and a **project deep dive**. HR round covers notice period, relocation, shifts and salary.
Product and GCC	Amazon, Microsoft, Adobe, Walmart, Flipkart, Oracle, SAP, Atlassian	Medium DSA (arrays, strings, hash maps, trees), JS polyfills, React machine coding, basic system design, behavioural questions (Amazon: leadership principles).
Startups and mid-size	Product startups, SaaS companies	Hands-on take-home or live coding, API design, debugging, ownership and speed.

Interview-day checklist

- Revise your resume line by line: every tool, number and responsibility can be questioned.
- Prepare two project stories (one front end, one back end or AI) with challenge, decision and result.
- Practise 10 JavaScript output questions, 5 SQL queries and 3 React machine-coding tasks.
- Think aloud: clarify the problem, state your approach, write code, test with an example, then discuss complexity.
- If you do not know an answer, say what you know, explain how you would find out, and never bluff.
- Keep a stable internet connection, a working camera and microphone, and a code editor ready for online rounds.
- After each interview, write down the questions you were asked and improve those topics.

Good luck! Consistent practice of fundamentals, honest project explanations and clear communication matter more than memorised answers.